VIT Agentic AI Workshop Curriculum

Title: Agentic AI in Practice: Build, Review and Control
Speaker: Vedant
Audience: First- to fourth-year IT/CS students, new to agentic AI
When: Saturday 17 October 2026, 11:00-13:00 IST
Duration: 120 minutes, including a five-minute break

Outcomes and participation

Each student aims to write a concrete coding-assistant instruction, make and run a small change in a prepared program, explain its result, and identify an unchecked value in an AI-generated answer. These are introductory outcomes. The classroom deck has 31 slides; full graph construction and the longer reference material remain optional study.

Plan for a possible 100-200 students; actual attendance, laptop availability and support are not confirmed. Students use their own laptop where available. Sharing is optional, with turns at the keyboard and individual notes. The presenter teaches from the front using short demonstrations and common checkpoints, with no requirement to inspect every laptop.

The current live path is starter/README.md in the GitHub kit. Use its notes template. Dependencies should be installed before class. Student activities use --offline; the instructor may separately demonstrate a privately tested live model. Coding-assistant accounts and live API access are optional.

Two-hour schedule

Time IST	Minutes	Block	Student outcome
11:00-11:10	10	Canteen story and model/tool/code	Trace who requests, runs and checks
11:10-11:20	10	Instructor API walkthrough and Checkpoint 0	Open a working starter or saved trace
11:20-11:40	20	Project context and Markdown	Checkpoint 1: write a useful instruction
11:40-12:00	20	One agent and one small change	Checkpoint 2: change, run and explain
12:00-12:05	5	Break	Resume at 12:05
12:05-12:25	20	Review the final answer	Checkpoint 3: find an unchecked value
12:25-12:40	15	Read a graph and choose control	Follow a route and discuss boundaries
12:40-13:00	20	Individual exit note and Q&A	Checkpoint 4 by 12:50, then questions

The slide script uses the same deadlines. Student tasks occupy at least 24 minutes, with ten minutes reserved for closing questions. Rehearse with a timer and preserve the activities if extra explanations run long.

One canteen example

The fictional menu has an available vegetarian sandwich at Rs 60, sold-out poha at Rs 35, an available chicken roll at Rs 75 and a vegetarian dosa at Rs 90. The normal request is vegetarian lunch under Rs 80 plus hours. The mock clock is 12:30; closing time is 16:00. No orders or payments occur.

Begin with a deliberately invented Paneer Tikka Wrap. Explain model request, application execution and tool result before framework names. Label the rule-based offline stand-in and injected mistakes honestly.

Five common checkpoints

1. **Checkpoint 0, by 11:20:** run starter/agent.py with --offline. Identify the sandwich and closing time. If blocked, open the supplied saved trace and record trace-reading rather than a local run.

2. **Checkpoint 1, by 11:36:** each student writes five concrete instruction lines in a scratch note, then adds the best line to supplied starter/AGENTS.md without removing guidance. Explain how to check that instruction. Briefly teach a heading, bullets and one command block; distinguish instructions, data and runtime code.

3. **Checkpoint 2, by 12:00:** copy starter/agent.py to starter/practice_agent.py. Change only QUESTION from Rs 80 to Rs 30, run the copy with --offline and explain why no dish qualifies while hours and stock remain unchanged. Keep tools, data and MENU_ONLY_QUESTION intact. Early finishers can try sold_out or menu_timeout.

4. **Checkpoint 3, by 12:18:** each student reviews docs/REVIEW.md, identifies the unchecked message and writes an input/output check. Optional neighbour discussion follows. The instructor demonstrates corrected demo 04; students do not build the whole graph live.

5. **Checkpoint 4, by 12:50:** each student saves a note containing one instruction, a changed line and result, and a review finding. Label predictions as not run. Sample explanations across the room; individual presenter approval is not required.

Room support and fallback

Repeat demonstrate, do, check. Students signal 1 finger ready, 2 need help, 3 using saved trace. Request 4-6 technical volunteers for seating zones after they have tried the starter; staffing is not confirmed. Helpers handle individual issues while the presenter explains repeated problems from the front.

Without helpers, individual installation support cannot be guaranteed. Blocked students use source, menu, saved trace or optional sharing. They write a proposed edit and label predicted results not run, completing execution afterward. Keep the instruction/review tasks moving; do not inspect every laptop or create an approval queue.

Distribute the exact ZIP before class; a private repository link alone does not grant access. Confirm attendance, laptops, Wi-Fi, power, projector and volunteers. See docs/ROOM_GUIDE.md for delivery details.

API walkthrough and framework scope

From the front, demonstrate AI Studio sign-in, API Keys and free-tier/model access using blank or masked configuration. Personal setup is private, outside the activity; avoid simultaneous audience account creation or live calls. AI Studio is 18+; access and quota vary, and core tasks run offline. No Cloud Console provisioning, billing upgrade or credit purchase. Test the actual live agent separately.

LangGraph carries state through nodes and routes and can contain flexible agents. ReAct is a pattern; CrewAI provides crews and flows. Framework choice alone does not determine autonomy. Full graph construction, multi-agent implementation, CrewAI setup, the Ainaa clip and deployment are optional follow-up. The preceding speaker covers Azure, a LangChain agent and MCP.

Official references

API keys, free tier, access requirements, LangGraph, CrewAI crews and flows.